

Home Lab Server Setup Guide

Dell OptiPlex 3050 Micro Server Setup (Ubuntu Server, Ollama, Open WebUI, n8n, VS Code, Nginx)

Target Hardware Specifications:

CPU: Intel Core i5-7500T (2.7 GHz)

RAM: 8GB DDR4 (16GB Recommended)

Storage: 256GB NVMe SSD

Network: Gigabit Ethernet / Wi-Fi

Step 1: Requirements & Prerequisites

To begin setting up your home lab server, prepare the following items:

- **Pendrive / USB Drive:** Minimum 8GB capacity.
- **Monitor & Keyboard:** Required only for initial installation.
- **Ethernet Cable:** LAN connection recommended for server stability.
- **Software Tools:** Rufus (for creating bootable USB) and Ubuntu Server 24.04 LTS ISO image.

Step 2: Installing Ubuntu Server OS

1. **Download OS & Rufus:** Download **Ubuntu Server 24.04 LTS** ISO from the official website and download **Rufus** on a separate PC.
2. **Create Bootable USB:** Open Rufus, select your USB drive, choose the downloaded Ubuntu ISO, and click **START**.
3. **Boot Settings:** Insert the USB into the Dell OptiPlex 3050. Turn on the PC and press **F12** to open the Boot Menu. Select the USB drive.
4. **Installation Steps:**
 - Choose Language and Keyboard layout.
 - Select **Ubuntu Server (Minimal/Standard)**.
 - **Network Configuration:** Note the assigned IP address (e.g., **192.168.0.100**).
 - **Storage Setup:** Choose "Use an entire disk" (256GB SSD).
 - **User Setup:** Create username (e.g., **admin**) and server name (e.g., **homelab**).
 - **OpenSSH Server:** Make sure to check/enable **Install OpenSSH Server**.
5. Finish installation, unplug the USB, and reboot. You can now disconnect the monitor and keyboard.

Step 3: Initial Server Access & System Updates

From your main PC or laptop, open Terminal / Command Prompt and connect via SSH:

```
ssh admin@192.168.0.100
```

Update system packages and install essential utility tools:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y curl wget git build-essential ufw
```

Step 4: Installing Docker & Docker Compose

Docker will be used to run services like Open WebUI, n8n, and VS Code in isolated containers.

```
# Install Docker automatic script  
curl -fsSL https://get.docker.com -o get-docker.sh  
sudo sh get-docker.sh  
  
# Enable Docker without sudo  
sudo usermod -aG docker $USER  
newgrp docker
```

Step 5: Installing Ollama & Llama 3.2 Model

Install Ollama directly on the host for optimal CPU performance:

```
curl -fsSL https://ollama.com/install.sh | sh
```

RAM Optimization Note: On an 8GB RAM system, run lightweight models such as `llama3.2:1b` or `llama3.2:3b` to keep memory usage low.

```
# Pull lightweight Llama 3.2 model  
ollama run llama3.2:3b
```

Step 6: Setting Up Docker Services (Open WebUI, n8n, VS Code Server)

Create a directory for Docker configurations and launch all core services using Docker Compose:

```
mkdir -p ~/homelab && cd ~/homelab  
nano docker-compose.yml
```

Paste the following content into `docker-compose.yml`:

```
version: '3.8'  
  
services:  
  # 1. Open WebUI (ChatGPT-style Interface for Ollama)  
  open-webui:  
    image: ghcr.io/open-webui/open-webui:main  
    container_name: open-webui  
    restart: always
```

```

ports:
  - "3000:8080"
extra_hosts:
  - "host.docker.internal:host-gateway"
environment:
  - OLLAMA_BASE_URL=http://host.docker.internal:11434
volumes:
  - open-webui-data:/app/backend/data

# 2. n8n Automation Tool
n8n:
  image: docker.n8n.io/n8nio/n8n:latest
  container_name: n8n
  restart: always
  ports:
    - "5678:5678"
  environment:
    - N8N_HOST=0.0.0.0
    - N8N_PORT=5678
    - N8N_PROTOCOL=http
  volumes:
    - n8n-data:/home/node/.n8n

# 3. VS Code Web Edition (code-server)
code-server:
  image: lscr.io/linuxserver/code-server:latest
  container_name: code-server
  restart: always
  ports:
    - "8443:8443"
  environment:
    - PUID=1000
    - PGID=1000
    - PASSWORD=YourStrongPasswordHere
  volumes:
    - code-server-config:/config
    - ~/projects:/config/workspace

# 4. Local Web Server (Nginx)
web-server:
  image: nginx:alpine
  container_name: web-server
  restart: always
  ports:
    - "8080:80"
  volumes:
    - ~/website:/usr/share/nginx/html:ro

volumes:
  open-webui-data:
  n8n-data:
  code-server-config:

```

Start all container services:

```
docker compose up -d
```

Step 7: Accessing Your Web Services

Open any web browser on your PC or mobile phone connected to the local network:

Service Name	Port	Access URL (Example)
Open WebUI (Llama UI)	3000	<code>http://192.168.0.100:3000</code>
n8n Workflows	5678	<code>http://192.168.0.100:5678</code>
VS Code Server	8443	<code>http://192.168.0.100:8443</code>
Local Web Hosting	8080	<code>http://192.168.0.100:8080</code>

Step 8: Resource Management & Upgrades

Recommended Upgrade: 8GB RAM will run Ubuntu + Docker + Ollama (3B Model), but upgrade to **16GB RAM** to ensure smooth multi-user handling and execution of 7B/8B AI models.

- Monitor resource consumption using: `htop` or `docker stats`
- Host local static HTML/CSS files in `~/website` directory to serve them via Nginx.